

Security Audit Report: OWASP Juice Shop

Auditor	<YOUR NAME>
Program	VortexTech Cyber Security Internship, Week 3
Target	OWASP Juice Shop (bkimminich/juice-shop , Docker) at http://localhost:3000
Date	<DATE>
Type	Black-box, manual + tool-assisted, beginner-level
Tools	Firefox/Chrome DevTools (Elements, Network, Console), OWASP ZAP, curl

1. Scope and Rules

- In scope: the local Juice Shop instance only.
- Out of scope: any real/production website, denial-of-service, attacks on the host machine.
- Juice Shop is intentionally vulnerable and designed for legal testing.

2. Methodology

1. Explored the app as a normal user (browse, register, log in, basket, reviews).

2. Tested inputs (search bar, login form) with unusual characters and payloads.
3. Inspected requests/responses in DevTools → Network for sensitive data.
4. Probed common paths (/ftp , /metrics) and API endpoints.
5. Ran ZAP's automated scan and cross-checked its alerts with manual results.
6. Mapped each finding to the OWASP Top 10 (2021) and wrote a remediation.

3. Summary of Findings

#	Finding	OWASP Top 10 (2021)	Severity
1	DOM-based XSS in search	A03 Injection	High
2	SQL injection authentication bypass	A03 Injection	Critical
3	Weak/guessable admin password	A07 Identification & Auth Failures	High
4	Exposed /ftp directory with confidential files	A05 Security Misconfiguration	High
5	Verbose error messages / stack traces	A05 Security Misconfiguration	Medium
6	Unauthenticated /metrics endpoint	A05 Security Misconfiguration	Medium

#	Finding	OWASP Top 10 (2021)	Severity
7	Basket IDOR (access other users' baskets)	A01 Broken Access Control	High

Categories covered: **Injection (XSS + SQLi), Broken Authentication, Security Misconfiguration / Sensitive Data Exposure, Broken Access Control**, which exceeds the required 3.

4. Detailed Findings

Finding 1: DOM-Based Cross-Site Scripting (XSS) in Search

- **Category:** A03 Injection (XSS) | **Severity:** High
- **Location:** Search bar → `http://localhost:3000/#/search?q=`

How I found it

1. Clicked the magnifier icon and searched for `<script>alert(1)</script>`. No alert (script tags inserted via `innerHTML` do not execute), but the input was reflected in the page, so I suspected unsafe rendering.
2. Searched with an event/iframe-based payload:

```
<iframe src="javascript:alert(`xss`)">
```

3. An alert box appeared, confirming JavaScript execution.
4. DevTools → Elements showed the payload injected unsanitised into the DOM. The payload also lives in the URL, so it can be shared as a malicious link.

Impact

An attacker can craft a link that runs JavaScript in a victim's browser: session/token theft (the JWT is stored client-side), account takeover, defacement, phishing overlays.

Remediation

- Never render user input with `innerHTML` / `bypassSecurityTrustHtml` ; use framework-safe bindings (Angular `{{ }}` , `textContent`).
- Contextually encode output; sanitise with a vetted library (e.g. `DOMPurify`) if HTML is required.
- Add a strict `Content-Security-Policy` (no `unsafe-inline`), and set cookies `HttpOnly` .

Evidence: `screenshots/01-xss-alert.png` , `screenshots/01-xss-dom.png`

Finding 2: SQL Injection Authentication Bypass

- **Category:** A03 Injection | **Severity:** Critical
- **Location:** `POST /rest/user/login`

How I found it

1. On the login page, entered a single quote `'` as the email with any password. The app returned an error (see Finding 5), hinting at unsanitised SQL.
2. Logged in with:
 - Email: `' OR 1=1 --`
 - Password: anything
3. Login succeeded as the first user in the database (the administrator account).

4. DevTools → Network confirmed the request body sent the payload verbatim and the response contained a valid JWT.

Impact

Complete authentication bypass and access to any account, including admin. Attackers can also use SQLi to read or modify the database (user data, orders).

Remediation

- Use parameterised queries / prepared statements or an ORM with bound parameters, never string concatenation.
- Validate input server-side (type, length, format).
- Run the DB account with least privilege; add a WAF as defence in depth only.

Example (Node/Sequelize):

```
// Vulnerable
sequelize.query(`SELECT * FROM Users WHERE
email='${email}' AND password='${hash}'`)
// Fixed
sequelize.query('SELECT * FROM Users WHERE email =
:email AND password = :hash',
  { replacements: { email, hash }, type:
QueryTypes.SELECT })
```

Evidence: screenshots/02-sqli-login.png , screenshots/02-network-jwt.png

Finding 3: Weak / Guessable Administrator Password

- **Category:** A07 Identification and Authentication Failures |
Severity: High
- **Location:** Login page

How I found it

1. Identified the admin email (`admin@juice-sh.op`) from product reviews on the site.
2. Tried common passwords. `admin123` was accepted.
3. Note: the login error message ("Invalid email or password.") is generic and does **not** reveal whether the account exists, which is a good practice. However, there was no visible rate limiting or lockout on repeated attempts.

Impact

Trivial credential guessing/brute force gives full admin access.

Remediation

- Enforce a strong password policy (length ≥ 12 , breached-password check, e.g. NIST 800-63B).
- Rate-limit and lock/slow down repeated failed logins; add CAPTCHA/MFA for admin accounts.
- Never use default or predictable credentials for privileged accounts.

Evidence: `screenshots/03-admin-login.png`

Finding 4: Exposed Directory Listing (`/ftp`) with Confidential Files

- **Category:** A05 Security Misconfiguration (Sensitive Data Exposure) | **Severity:** High
- **Location:** `http://localhost:3000/ftp`

How I found it

1. Browsed to `/ftp` (a typical path also flagged by ZAP's directory browsing / forced browsing checks).

2. The server returned an open directory listing containing internal documents (e.g. `acquisitions.md`, `legal.md`) and backup-type files, downloadable without authentication.

Impact

Disclosure of confidential business information and internal files; backup/config files can reveal further attack paths.

Remediation

- Disable directory listing; remove non-public files from the web root.
- Require authentication/authorisation for any file-serving route; allow-list file types.
- Regularly audit exposed paths.

Evidence: `screenshots/04-ftp-listing.png`

Finding 5: Verbose Error Messages Leak Internal Details

- **Category:** A05 Security Misconfiguration | **Severity:** Medium
- **Location:** Login and product search endpoints
(`/rest/products/search?q=`)

How I found it

1. Sent a lone `'` in the search query and in the login email field.
2. DevTools → Network showed a `500` response whose body contained a database error and a server-side stack trace (framework, file paths, SQL dialect).

Impact

Reveals technology stack, file structure, and query structure, which speeds up further attacks (it directly helped confirm Finding 2).

Remediation

- Return generic error messages to clients; log details server-side only.
- Set production mode (`NODE_ENV=production`) and use a global error handler.

Evidence: `screenshots/05-stack-trace.png`

Finding 6: Unauthenticated Monitoring Endpoint

(`/metrics`)

- **Category:** A05 Security Misconfiguration | **Severity:** Medium
- **Location:** `http://localhost:3000/metrics`

How I found it

1. Requested `/metrics` (a common Prometheus path; also surfaced during ZAP spidering/manual path probing).
2. Received Prometheus-format runtime and application metrics with no login.

Impact

Discloses internal behaviour, usage counters, and infrastructure details useful for reconnaissance and timing attacks.

Remediation

- Restrict monitoring endpoints to internal networks / require authentication.
- Do not expose them via the public reverse proxy.

Evidence: `screenshots/06-metrics.png`

Finding 7: Insecure Direct Object Reference (IDOR) on Baskets

- **Category:** A01 Broken Access Control | **Severity:** High
- **Location:** GET /rest/basket/{id}

How I found it

1. Logged in as a normal user, added an item to the basket, and watched the request in DevTools → Network: GET /rest/basket/6 (my basket ID).
2. Changed the ID (in the browser Storage `bid` value or via curl with my token) to another number, e.g. /rest/basket/1 .
3. The server returned another user's basket contents.

Impact

Any authenticated user can read (and potentially modify) other customers' data by iterating IDs.

Remediation

- Enforce server-side authorisation: verify the basket belongs to the authenticated user on every request.
- Use unpredictable identifiers (UUIDs) as defence in depth, never as the only control.

Evidence: screenshots/07-idor-basket.png

5. OWASP ZAP Results (summary)

Ran *Automated Scan* against `http://localhost:3000` . Typical alerts observed and how they relate to this report:

ZAP alert	Related finding / note
Content Security Policy (CSP) header not set	Finding 1 (no defence-in-depth vs XSS)

ZAP alert	Related finding / note
Missing Anti-clickjacking header	Additional hardening item
Cross-Domain Misconfiguration (permissive CORS)	Security misconfiguration
Information disclosure / error messages	Finding 5
Timestamp / server info leakage	Minor

Replace/adjust this table with your actual ZAP report (export as HTML → put in `screenshots/` or repo root).

6. Prioritised Recommendations

1. Fix injection flaws first (parameterised queries, output encoding, CSP).
2. Enforce server-side access control on every object/route.
3. Harden authentication (strong passwords, MFA, rate limiting).
4. Lock down configuration: disable directory listing, hide metrics, generic errors, security headers.
5. Add automated security testing (ZAP in CI) and regular reviews.

7. Limitations

Beginner-level audit of a deliberately vulnerable app; not exhaustive. Findings are reproducible in Juice Shop only and do not represent a real production system.

8. References

- OWASP Top 10 (2021): <https://owasp.org/Top10/>
- OWASP Juice Shop: <https://owasp.org/www-project-juice-shop/>
- OWASP ZAP: <https://www.zaproxy.org/>
- OWASP Cheat Sheets (SQL Injection Prevention, XSS Prevention, Authentication)